

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_191a9b86-991\agent-aeeb68b725d16a62b.jsonl`
- SHA-256: `b944f03c74ca5d2c457b4129a78b6b1f777fbf75cbce34148e947ac2e587fee7`
- Source modified: `2026-06-22T03:48:13+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_191a9b86-991`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T03:45:49.095Z)

Review the P2c first-run wizard for CORRECTNESS + EDGE CASES. Run `cd C:\\Users\\avidu\\Projects\\khelsutra-p2c && git diff origin/master...HEAD`` and read `web/src/components/SetupWizard.tsx` + `SetupWizard.test.tsx` + `web/src/api/{client,types}.ts` in `C:\\Users\\avidu\\Projects\\khelsutra-p2c`. Scrutinize: (1) the capabilities fetch + the loading/offline/stepping state machine ? any race (unmount during fetch, alive guard), or a state where buttons mislead? (2) `localStorage` flag (`WIZARD_DONE_KEY`) read/write + the try/catch (private mode); does `isWizardDone` behave right? (3) the optional-chaining over capabilities (`gemini_key_present` / `ffmpeg` / `gpu` / `default_segmenter`) ? does a partial/empty caps object degrade safely without crashing? (4) the Back/Next/Done/Skip transitions + the `isLast/disabled` logic. Report concrete findings (file:line) only; empty list if none.

2. assistant (2026-06-22T03:45:51.409Z)

I'll review the P2c first-run wizard. Let me start by pulling the repo state and reading the diff and relevant files.

3. user (2026-06-22T03:45:55.360Z)

```
---BRANCH---
feat/p2c-setup-wizard
---HEAD---
e6b9725 feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
---DIFFSTAT---
 locales/da/common.json          | 32 ++++++
 locales/en/common.json          | 32 ++++++
 locales/es/common.json          | 32 ++++++
 locales/hi/common.json          | 32 ++++++
 locales/id/common.json          | 32 ++++++
 locales/kn/common.json          | 32 ++++++
 web/src/App.tsx                  | 9 +-
 web/src/api/client.ts           | 6 ++
 web/src/api/types.ts            | 17 +++
 web/src/components/SetupWizard.test.tsx | 115 ++++++
 web/src/components/SetupWizard.tsx | 179 ++++++
 web/src/components/a4.touch.test.tsx | 4 +
12 files changed, 521 insertions(+), 1 deletion(-)
```

4. assistant (2026-06-22T03:45:57.434Z)

Let me read all the relevant files.

5. user (2026-06-22T03:45:58.551Z)

```

1      // First-run setup wizard (PACKAGING_PLAN.md P2c). Three quick steps ? AI key / compute
/ videos ?
2      // rendered from the engine's GET /api/capabilities (an honest probe of what THIS box
can do).
3      // Localized (en/kn/hi/es/da/id). The Gemini-key step is GUIDED setup, not auto-save:
the engine has
4      // no key-write endpoint yet, so we tell the user how to add the key (offline `motion`
works with none).
5      import { useEffect, useState } from "react";
6      import { useTranslation } from "react-i18next";
7      import { getCapabilities } from "../api/client";
8      import type { EngineCapabilities } from "../api/types";
9      import { log, newCorrelationId } from "../lib/log";
10
11     /** localStorage flag ? distinct from the language key ? so the wizard shows ONCE per
browser. */
12     export const WIZARD_DONE_KEY = "kheisutra.wizard.completed";
13
14     export function isWizardDone(): boolean {
15         try {
16             return localStorage.getItem(WIZARD_DONE_KEY) === "true";
17         } catch {
18             return false; // private-mode / no storage ? show it (harmless; user can skip)
19         }
20     }
21
22     const STEPS = 3;
23
24     function KeyStep({ caps }: { caps: EngineCapabilities | null }) {
25         const { t } = useTranslation();
26         const present = caps?.gemini_key_present === true;
27         return (
28             <div>
29                 <h3 className="font-semibold">{t("wizard.key.title")}</h3>
30                 {present ? (
31                     <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
32                 ) : (
33                     <>
34                         <p className="mt-2 text-ink/80">{t("wizard.key.missing")}</p>
35                         <p className="mt-2 text-sm text-ink/70">{t("wizard.key.how")}</p>
36                         <a
37                             href="https://aistudio.google.com/apikey"
38                             target="_blank"
39                             rel="noreferrer"
40                             className="mt-2 inline-block text-sm font-medium text-green underline"
41                         >
42                             {t("wizard.key.getKey")}
43                         </a>
44                     </>
45                 )}
46             </div>
47         );
48     }
49
50     function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
51         const { t } = useTranslation();
52         const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
53         const gpu = caps?.gpu?.nvidia_smi_present === true;
54         const detector = caps?.default_segmenter ?? "motion";
55         return (
56             <div>
57                 <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
58                 <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
59                 <ul className="mt-3 space-y-1 text-sm text-ink/70">
60                     <li>{t("wizard.compute.detector", { name: detector })}</li>

```

```

61         <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
t("wizard.compute.ffmpegMissing")}</li>
62         <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>
63     </ul>
64 </div>
65 );
66 }
67
68 function VideosStep() {
69     const { t } = useTranslation();
70     return (
71         <div>
72             <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
73             <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
74             <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
75         </div>
76     );
77 }
78
79 export function SetupWizard({ onClose }: { onClose: () => void }) {
80     const { t } = useTranslation();
81     const [step, setStep] = useState(0);
82     const [caps, setCaps] = useState<EngineCapabilities | null>(null);
83     const [loading, setLoading] = useState(true);
84     const [offline, setOffline] = useState(false);
85
86     useEffect(() => {
87         let alive = true;
88         const cid = newCorrelationId();
89         getCapabilities(cid)
90             .then((c) => {
91                 if (!alive) return;
92                 setCaps(c);
93                 setLoading(false);
94             })
95             .catch((err) => {
96                 if (!alive) return;
97                 // Engine not up yet ? don't assert false capabilities; offer a graceful skip.
98                 setOffline(true);
99                 setLoading(false);
100                 log("warn", "wizard.capabilities_unavailable", { cid, error: String(err) });
101             });
102         return () => {
103             alive = false;
104         };
105     }, []);
106
107     const finish = () => {
108         try {
109             localStorage.setItem(WIZARD_DONE_KEY, "true");
110         } catch {
111             /* private mode: still close (the wizard just reappears next visit) */
112         }
113         onClose();
114     };
115
116     const stepping = !loading && !offline; // the 3-step flow only when capabilities
loaded
117     const isLast = step === STEPS - 1;
118
119     return (
120         <section
121             role="dialog"
122             aria-label={t("wizard.title")}
123             className="mt-8 rounded-2xl border border-ink/10 bg-white p-6 shadow-sm"

```

```

124     >
125     <div className="flex items-center justify-between gap-3">
126       <h2 className="text-xl font-bold">{t("wizard.title")}</h2>
127       <button
128         type="button"
129         onClick={finish}
130         className="min-h-11 rounded-lg px-3 text-sm text-ink/60 hover:text-ink"
131       >
132         {t("wizard.skip")}
133       </button>
134     </div>
135     <p className="mt-1 text-sm text-ink/70">{t("wizard.intro")}</p>
136     {stepping && (
137       <p className="mt-1 text-xs font-medium text-ink/50">{t("wizard.step", { n: step
+ 1 })}</p>
138     )}
139
140     <div className="mt-5 min-h-[128px]" aria-live="polite">
141       {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142       {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143       {stepping && step === 0 && <KeyStep caps={caps} />}
144       {stepping && step === 1 && <ComputeStep caps={caps} />}
145       {stepping && step === 2 && <VideosStep />}
146     </div>
147
148     <div className="mt-6 flex items-center justify-between">
149       {stepping ? (
150         <>
151           <button
152             type="button"
153             disabled={step === 0}
154             onClick={() => setStep((s) => Math.max(0, s - 1))}
155             className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"
156           >
157             {t("wizard.back")}
158           </button>
159           <button
160             type="button"
161             onClick={() => (isLast ? finish() : setStep((s) => Math.min(STEPS - 1, s +
162             1)))}
163             className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
164           >
165             {isLast ? t("wizard.done") : t("wizard.next")}
166           </button>
167         </>
168       ) : (
169         <button
170           type="button"
171           onClick={finish}
172           className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
173         >
174           {t("wizard.done")}
175         </button>
176       )}
177     </div>
178   </section>
179 );
180 }

```

```

1 // @vitest-environment jsdom
2 import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
3 import { render, screen, fireEvent, cleanup, waitFor } from "@testing-library/react";
4 import i18n from "../i18n/index";
5 import { SetupWizard, WIZARD_DONE_KEY, isWizardDone } from "../SetupWizard";
6 import type { EngineCapabilities } from "../api/types";
7
8 function stubCapabilities(caps: EngineCapabilities | "reject") {
9   vi.stubGlobal(
10     "fetch",
11     vi.fn(async (url: string | URL) => {
12       const u = typeof url === "string" ? url : url.toString();
13       if (u.endsWith("/api/capabilities")) {
14         if (caps === "reject") throw new TypeError("Failed to fetch");
15         return { ok: true, status: 200, json: async () => caps } as unknown as Response;
16       }
17       return { ok: false, status: 404, text: async () => "" } as unknown as Response;
18     }),
19   );
20 }
21
22 const LIGHT: EngineCapabilities = {
23   gemini_key_present: false,
24   default_segmenter: "motion",
25   ffmpegpeg: { ffmpegpeg: false, ffprobe: false },
26   gpu: { nvidia_smi_present: false },
27 };
28 const READY: EngineCapabilities = {
29   gemini_key_present: true,
30   default_segmenter: "gemini",
31   ffmpegpeg: { ffmpegpeg: true, ffprobe: true },
32   gpu: { nvidia_smi_present: true },
33 };
34
35 beforeEach(async () => {
36   window.localStorage.clear();
37   await i18n.changeLanguage("en");
38 });
39 afterEach(() => {
40   cleanup();
41   vi.restoreAllMocks();
42   vi.unstubAllGlobals();
43 });
44
45 describe("SetupWizard", () => {
46   it("checks capabilities, then step 1 guides a missing Gemini key (offline-first default)", async () => {
47     stubCapabilities(LIGHT);
48     render(<SetupWizard onClose={() => {}} />);
49     expect(screen.getByText(/Checking what this device can do/)).toBeTruthy();
50     expect(await screen.findByText(/built-in motion detector/)).toBeTruthy();
51     expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
52     expect(screen.getByText(/Step 1 of 3/)).toBeTruthy();
53   });
54
55   it("walks all 3 steps from real capabilities; Back returns; Done sets the flag + closes", async () => {
56     stubCapabilities(LIGHT);
57     const onClose = vi.fn();
58     render(<SetupWizard onClose={onClose} />);
59     await screen.findByText(/built-in motion detector/);
60
61     fireEvent.click(screen.getByText("Next")); // ? compute
62     expect(screen.getByText("Detector: motion")).toBeTruthy();
63     expect(screen.getByText(/ffmpegpeg not found/)).toBeTruthy();

```

```

64     expect(screen.getByText(/No GPU needed/)).toBeTruthy();
65
66     fireEvent.click(screen.getByText("Back")); // ? key
67     expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
68
69     fireEvent.click(screen.getByText("Next")); // ? compute
70     fireEvent.click(screen.getByText("Next")); // ? videos
71     expect(screen.getByText(/You're all set/)).toBeTruthy();
72
73     fireEvent.click(screen.getByText("Get started")); // Done on last step
74     expect(onClose).toHaveBeenCalledTimes(1);
75     expect(window.localStorage.getItem(WIZARD_DONE_KEY)).toBe("true");
76 });
77
78 it("reflects a connected key + a GPU/ffmpeg-ready box", async () => {
79     stubCapabilities(READY);
80     render(<SetupWizard onClose={() => {}} />);
81     expect(await screen.findByText(/Gemini is connected/)).toBeTruthy();
82     fireEvent.click(screen.getByText("Next"));
83     expect(screen.getByText("Detector: gemini")).toBeTruthy();
84     expect(screen.getByText(/Video tools \(\ffmpeg\) ready/)).toBeTruthy();
85     expect(screen.getByText(/A GPU was detected/)).toBeTruthy();
86 });
87
88 it("Skip closes + sets the flag without finishing the steps", async () => {
89     stubCapabilities(LIGHT);
90     const onClose = vi.fn();
91     render(<SetupWizard onClose={onClose} />);
92     await screen.findByText(/built-in motion detector/);
93     fireEvent.click(screen.getByText("Skip"));
94     expect(onClose).toHaveBeenCalledTimes(1);
95     expect(isWizardDone()).toBe(true);
96 });
97
98 it("degrades gracefully when the engine is unreachable (offline) ? message + a Done
escape", async () => {
99     stubCapabilities("reject");
100     const onClose = vi.fn();
101     render(<SetupWizard onClose={onClose} />);
102     expect(await screen.findByText(/Couldn't reach KhelSutra/)).toBeTruthy();
103     // No step content / step indicator when offline.
104     expect(screen.queryByText(/Step 1 of 3/)).toBeNull();
105     fireEvent.click(screen.getByText("Get started"));
106     await waitFor(() => expect(onClose).toHaveBeenCalledTimes(1));
107     expect(isWizardDone()).toBe(true);
108 });
109
110 it("isWizardDone reads the localStorage flag", () => {
111     expect(isWizardDone()).toBe(false);
112     window.localStorage.setItem(WIZARD_DONE_KEY, "true");
113     expect(isWizardDone()).toBe(true);
114 });
115 });
116

```

7. user (2026-06-22T03:45:59.821Z)

```

1     // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2     // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3     // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4     import { log, newCorrelationId } from "../lib/log";
5     import type {

```

```

6      RallySegment,
7      QueryResponse,
8      CompileResponse,
9      EngineConfig,
10     EngineCapabilities,
11     ProcessResponse,
12     JobStatus,
13     UploadInitResponse,
14     UploadPartResponse,
15     UploadCompleteResponse,
16 } from "./types";
17
18 /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
19 * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
20 * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
21 * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
22 * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
23 * `` `${base}/process` `` never double-slashes. */
24 export function apiBase(): string {
25     return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
26 }
27
28 const BASE = apiBase();
29
30 export class ApiError extends Error {
31     readonly cid?: string;
32     readonly status?: number;
33     readonly body?: string;
34     constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
35         super(message);
36         this.name = "ApiError";
37         this.cid = opts.cid;
38         this.status = opts.status;
39         this.body = opts.body;
40         if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
41     }
42 }
43
44 interface RequestOpts {
45     method?: string;
46     body?: string;
47     headers?: Record<string, string>;
48     /** Correlation id to thread through this call; auto-generated if absent. */
49     cid?: string;
50 }
51
52 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
53     const cid = opts.cid ?? newCorrelationId();
54     const url = `${BASE}${path}`;
55     const method = opts.method ?? "GET";
56     log("info", "api.request", { cid, method, url });
57
58     let res: Response;
59     try {
60         res = await fetch(url, {
61             method,
62             body: opts.body,
63             headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },

```

```

64     });
65     } catch (err) {
66         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
67         log("error", "api.network_error", { cid, method, url, error: String(err) });
68         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
69     }
70
71     if (!res.ok) {
72         const body = await res.text().catch(() => "");
73         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
74         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
75     }
76
77     const data = (await res.json()) as T;
78     log("info", "api.response", { cid, method, url, status: res.status });
79     return data;
80 }
81
82 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
83 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
84 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
85 export async function processVideo(videoPath: string, cid?: string, locale?: string):
Promise<ProcessResponse> {
86     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). Optional
87     // + omitted when absent (JSON.stringify drops undefined); the engine ignores it until
wired.
88     return request<ProcessResponse>("/process", {
89         method: "POST",
90         body: JSON.stringify({ video_path: videoPath, locale }),
91         cid,
92     });
93 }
94
95 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
96 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
97     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
98 }
99
100 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
101 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
102     const data = await request<QueryResponse>("/query", {
103         method: "POST",
104         body: JSON.stringify({ video_id: videoId }),
105         cid,
106     });
107     return data.play_segments ?? [];
108 }
109
110 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
111 export async function compileReel(
112     videoId: string,
113     segmentIds: number[],
114     outputFilename: string,
115     cid?: string,
116     locale?: string,

```

```

117     ): Promise<CompileResponse> {
118         // `locale` (the UI language) localizes the reel's branding watermark engine-side.
Optional +
119         // omitted when absent; the engine falls back to the default watermark for an
unknown/absent locale.
120         return request<CompileResponse>("/compile", {
121             method: "POST",
122             body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
123             cid,
124         });
125     }
126
127     /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
128     export function highlightUrl(videoId: string, filename: string): string {
129         return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
130     }
131
132     /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
133     export async function getConfig(cid?: string): Promise<EngineConfig> {
134         return request<EngineConfig>("/config", { method: "GET", cid });
135     }
136
137     /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
138     export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
139         return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
140     }
141
142     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
143     export function minShotCount(cfg: EngineConfig): number | undefined {
144         return cfg.stitching?.min_shot_count;
145     }
146
147     // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
148     // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
149     // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
150
151     /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
152     export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
153         return request<UploadInitResponse>("/upload/init", {
154             method: "POST",
155             body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
156             cid,
157         });
158     }
159
160     /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
161     export async function uploadPart(
162         uploadId: string,
163         index: number,
164         chunk: Blob,
165         cid?: string,
166         signal?: AbortSignal,
167     ): Promise<UploadPartResponse> {
168         const c = cid ?? newCorrelationId();
169         const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;

```

```

170     log("info", "api.request", { cid: c, method: "PUT", url });
171     let res: Response;
172     try {
173         res = await fetch(url, {
174             method: "PUT",
175             body: chunk,
176             headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
177             signal,
178         });
179     } catch (err) {
180         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
181     });
182         throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
183     }
184     if (!res.ok) {
185         const body = await res.text().catch(() => "");
186         log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
187     body: body.slice(0, 500) });
188         throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
189     body });
190     }
191     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
192     return (await res.json()) as UploadPartResponse;
193 }
194
195 /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
196 */
197 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
198 string): Promise<UploadCompleteResponse> {
199     return
200     request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
201         method: "POST",
202         body: JSON.stringify({ total_parts: totalParts }),
203         cid,
204     });
205 }
206
207 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
208 [uploads.py:147]. */
209 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
210     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
211     cid });
212 }
213
214 export interface UploadProgress {
215     sentBytes: number;
216     totalBytes: number;
217 }
218
219 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
220 complete.
221 * On ANY failure or abort it best-effort DELETES the server-side partial, then
222 rethrows so the
223 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
224 video_id}.
225 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
226 recovery is a
227 * fresh init (re-pick the file), which the UI's retry does. */
228 export async function uploadFile(
229     file: File,
230     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
231     = {},
232 ): Promise<UploadCompleteResponse> {
233     const cid = opts.cid ?? newCorrelationId();
234     const init = await uploadInit(file.name, file.size, cid);

```

```

222      // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
223      // would silently upload zero parts) ? fall back to the engine's 95 MB default.
224      const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
225      const partBytes = partMb * 1024 * 1024;
226      const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
227      log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
228      let sent = 0;
229      try {
230        for (let i = 0; i < totalParts; i++) {
231          if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
232          const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
233          await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
234          sent += chunk.size;
235          opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
236        }
237        const done = await uploadComplete(init.upload_id, totalParts, cid);
238        log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
239        return done;
240      } catch (err) {
241        log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
242        await uploadAbort(init.upload_id, cid).catch(() => {});
243        throw err;
244      }
245    }
246

```

8. user (2026-06-22T03:46:00.016Z)

```

1      // Types mirror the engine's verified contract (badminton-highlight-indexer).
2      // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3      // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5      export interface RallyMetadata {
6        /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7        shot_count?: number;
8        /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9        shot_count_confidence?: string;
10       detector?: string;
11       [key: string]: unknown;
12     }
13
14     export interface RallySegment {
15       /** The integer the UI selects and passes to /api/compile as segment_ids. */
16       id: number;
17       start_time: number;
18       end_time: number;
19       duration: number;
20       /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21       server: string | null;
22       /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23       receiver: string | null;
24       metadata: RallyMetadata;
25       /** motion-only fabricated sub-events ? DO NOT surface. */
26       events?: unknown[];
27     }
28
29     export interface QueryResponse {
30       play_segments: RallySegment[];

```

```

31     }
32
33     export interface CompileResponse {
34         status: string;
35         /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36         filepath: string;
37     }
38
39     export interface EngineConfig {
40         stitching?: {
41             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42             */
43             min_shot_count?: number;
44             [key: string]: unknown;
45         };
46         [key: string]: unknown;
47     }
48     /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
49     rendered by
50     * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-booleans only (never the Gemini
51     key value).
52     * Every field optional/defensive: a missing field degrades the wizard gracefully. */
53     export interface EngineCapabilities {
54         api_version?: number;
55         engine_version?: string;
56         segmenters?: string[];
57         default_segmenter?: string;
58         ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
59         string };
60         gpu?: { nvidia_smi_present?: boolean };
61         /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
62         */
63         gemini_key_present?: boolean;
64         wasb_weights_present?: boolean;
65         deployment?: { profile?: string; compute_target?: string };
66         [key: string]: unknown;
67     }
68
69     /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
70     /api/jobs/{job_id}. */
71     export interface ProcessResponse {
72         job_id: string;
73         /** Always "queued" at submit. */
74         status: string;
75         /** Relative poll path, e.g. "/api/jobs/<id>". */
76         poll: string;
77     }
78
79     /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
80     sequential parts
81     * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
82     POST /api/process. */
83     export interface UploadInitResponse {
84         upload_id: string;
85         /** Per-part ceiling in MB ? slice the file into parts this big. */
86         max_part_mb: number;
87         next_part: string;
88     }
89
90     export interface UploadPartResponse {
91         upload_id: string;
92         received_parts: number;
93         received_bytes: number;
94     }
95
96     export interface UploadCompleteResponse {

```

```

88     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
89     path: string;
90     video_id: string;
91     size_bytes: number;
92   }
93
94   /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
95    * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
96    (success|failed).
97    * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
98    shaped
99    * FAILURE ? the UI must not treat it as a load. */
100   export interface JobStatus {
101     job_id: string;
102     status: "queued" | "running" | "done" | "failed";
103     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
104     percentage. */
105     progress?: string | null;
106     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
107     */
108     video_id?: string | null;
109     /** Execution error (worker crash / unexpected) when status === "failed". */
110     error?: string | null;
111     result?: { status?: string; video_id?: string; message?: string } | null;
112   }

```

9. assistant (2026-06-22T03:46:02.322Z)

Now let me look at the diff for App.tsx to understand how the wizard is mounted/gated, and the full diff context.

10. user (2026-06-22T03:46:03.785Z)

```

diff --git a/web/src/App.tsx b/web/src/App.tsx
index a0a7882..c516153 100644
--- a/web/src/App.tsx
+++ b/web/src/App.tsx
@@ -8,6 +8,7 @@ import { SelectionFooter } from "../components/SelectionFooter";
import { LanguageSwitcher } from "../components/LanguageSwitcher";
import { CompileResult, type BuildState } from "../components/CompileResult";
import { IngestPanel } from "../components/IngestPanel";
+import { SetupWizard, isWizardDone } from "../components/SetupWizard";
import { runQuery, type ParsedQuery } from "../lib/query";
import { floorRisk } from "../lib/floor";
import { reelFilename } from "../lib/format";
@@ -60,6 +61,8 @@ export default function App() {
  const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" } : {
status: "demo" });
  const [reelName, setReelName] = useState("highlights");
  const [build, setBuild] = useState<BuildState>({ status: "idle" });
+ // P2c: the first-run wizard shows once per browser, before the ingest panel, when nothing's
+ loaded.
+ const [wizardDone, setWizardDone] = useState(isWizardDone);

  // Display order is separate from selection: a query can re-sort the cards while the shared
  // selection set (sel) stays the single source of truth for what lands in the reel.
@@ -234,9 +237,13 @@ export default function App() {
    <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
  </p>

+    /** P2c first-run wizard: before anything is loaded, walk a new user through AI-key /
+    compute /
+    videos (from /api/capabilities). Shows once per browser; then the ingest panel
+    takes over. */

```

```

+      {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
+
+      {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a bucket URI /
host
      path ? process ? load. Once a video_id exists, the existing back-half takes over.
*/}
-      {!videoId && (
+      {!videoId && wizardDone && (
        <IngestPanel
          onLoad={({id}) => {
            setVideoId(id);
diff --git a/web/src/components/a4.touch.test.tsx b/web/src/components/a4.touch.test.tsx
index 1129650..4638b0f 100644
--- a/web/src/components/a4.touch.test.tsx
+++ b/web/src/components/a4.touch.test.tsx
@@ -11,6 +11,7 @@ import { render, screen, cleanup, fireEvent } from "@testing-library/react";
import i18n from "../i18n/index";
import App from "../App";
import { QueryBar } from "../QueryBar";
+import { WIZARD_DONE_KEY } from "../SetupWizard";

// 44px floor: min-h-11 (buttons/inputs) or an explicit h-11 square (the ? dismiss).
const TAP_44 = /\b(?:min-h-11|h-11)\b/;
@@ -18,6 +19,9 @@ const TAP_44 = /\b(?:min-h-11|h-11)\b/;
beforeEach(async () => {
  vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
  window.localStorage.clear();
+ // Skip the first-run wizard (P2c) so these tests render the app proper (IngestPanel + grid)
+ whose
+ // touch targets they assert ? the wizard's own ?44px targets are covered in
SetupWizard.test.tsx.
+ window.localStorage.setItem(WIZARD_DONE_KEY, "true");
  await i18n.changeLanguage("en");
});

```

11. assistant (2026-06-22T03:46:07.075Z)

Now let me check the locale file to verify the i18n keys used exist, and check the `log`/`newCorrelationId` lib and how `getCapabilities` errors surface. Let me also look at the `wizard.step` interpolation and the i18n key set.

12. user (2026-06-22T03:46:08.438Z)

```

diff --git a/locales/en/common.json b/locales/en/common.json
index aeaa01a..45bc8be 100644
--- a/locales/en/common.json
+++ b/locales/en/common.json
@@ -288,6 +288,38 @@
    }
  },
+  "wizard": {
+    "title": "Welcome to KhelSutra",
+    "intro": "Three quick steps and you're ready to turn a match into a highlights reel.",
+    "step": "Step {n} of 3",
+    "checking": "Checking what this device can do?",
+    "offline": "Couldn't reach KhelSutra ? start it, then reopen setup. You can skip for now.",
+    "back": "Back",
+    "next": "Next",
+    "skip": "Skip",
+    "done": "Get started",
+    "key": {
+      "title": "AI highlights (optional)",
+      "connected": "Gemini is connected ? you'll get AI-assisted highlights.",
+      "missing": "No AI key yet ? that's fine. KhelSutra works offline with the built-in motion

```

```

detector.",
+   "how": "Want sharper highlights? Get a free Gemini key, add GEMINI_API_KEY=? to the .env
file in your KhelSutra folder, then restart.",
+   "getKey": "Get a free Gemini key"
+ },
+   "compute": {
+     "title": "Where it runs",
+     "local": "Everything runs on this device ? your videos never leave it.",
+     "detector": "Detector: {name}",
+     "ffmpegOk": "Video tools (ffmpeg) ready.",
+     "ffmpegMissing": "ffmpeg not found ? install it so KhelSutra can cut and join clips.",
+     "gpu": "A GPU was detected ? the high-quality shuttle tracker can run here later.",
+     "noGpu": "No GPU needed ? the offline motion detector runs on any computer."
+   },
+   "videos": {
+     "title": "Add a video",
+     "body": "When you're ready, pick a video file or paste a cloud link and KhelSutra finds
the rallies for you.",
+     "ready": "You're all set."
+   }
+ },
+   "selection": {
+     "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
+   }
}

```

13. assistant (2026-06-22T03:46:11.596Z)

Now I notice the i18n interpolation uses `{n}` and `{name}` syntax. Let me check how i18n is configured ? specifically whether it uses default `{{var}}` interpolation or ICU/custom. This is critical: if i18next default interpolation is `{{var}}`, then `{n}` and `{name}` would not interpolate.

14. user (2026-06-22T03:46:11.912Z)

```

1      // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2      //
3      // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4      //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5      //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6      //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7      //   - one negotiation order shared with the marketing site (B3);
8      //   - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9      import i18n from "i18next";
10     import { initReactI18next } from "react-i18next";
11     import LanguageDetector from "i18next-browser-languagedetector";
12     import ICU from "i18next-icu";
13     // Catalogs are HOISTED to repo-root /locales (D-CATALOG-HOME) ? the single source of
truth shared
14     // by this SPA and the marketing site's t() shim (tracker B3). Vite resolves the
workspace root via
15     // the repo `.git` marker, so importing the parent-dir /locales works in dev, build, and
viteest.
16     import enCommon from "../../locales/en/common.json";
17     import knCommon from "../../locales/kn/common.json";
18     import hiCommon from "../../locales/hi/common.json";
19     import esCommon from "../../locales/es/common.json";
20     import daCommon from "../../locales/da/common.json";
21     import idCommon from "../../locales/id/common.json";
22
23     // Supported locales: en/kn/hi foundation (FRIENDLY D6) + MULTILINGUAL_EXPANSION Phase A
(es/da/id,
24     // Latin, catalog-only). Anything else falls back to `en`.
25     export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;

```

```

26     export type Locale = (typeof SUPPORTED_LOCALES)[number];
27     export const DEFAULT_LOCALE: Locale = "en";
28
29     // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
30     // visitor's language choice carries across both surfaces.
31     export const LANG_STORAGE_KEY = "khelsutra.lang";
32
33     // Catalogs live in repo-root /locales (D-CATALOG-HOME), consumed by both surfaces. `kn`
+ `hi`
34     // shipped (B5/B6, AI-drafted ? native review pending). A missing key in any locale
still falls back
35     // to `en` ? the by-design invariant.
36     const resources = {
37       en: { common: enCommon },
38       kn: { common: knCommon },
39       hi: { common: hiCommon },
40       es: { common: esCommon },
41       da: { common: daCommon },
42       id: { common: idCommon },
43     } as const;
44
45     void i18n
46       .use(ICU)
47       .use(LanguageDetector)
48       .use(initReactI18next)
49       .init({
50         resources,
51         fallbackLng: DEFAULT_LOCALE,
52         supportedLngs: [...SUPPORTED_LOCALES],
53         nonExplicitSupportedLngs: true, // en-IN -> en
54         defaultNS: "common",
55         ns: ["common"],
56         interpolation: { escapeValue: false }, // React already escapes
57         returnNull: false,
58         // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
59         detection: {
60           order: ["localStorage", "querystring", "navigator"],
61           lookupLocalStorage: LANG_STORAGE_KEY,
62           lookupQuerystring: "lang",
63           caches: ["localStorage"],
64         },
65         react: { useSuspense: false },
66       });
67
68     // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
69     // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
70     function syncDocumentLang(lng: string) {
71       if (typeof document !== "undefined") {
72         document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
73       }
74     }
75     i18n.on("languageChanged", syncDocumentLang);
76     syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
77
78     export default i18n;
79

```

15. assistant (2026-06-22T03:46:15.640Z)

Good ? it uses `i18next-icu` (ICU MessageFormat), so `{n}` and `{name}` single-brace interpolation is correct. The tests confirm "Step 1 of 3" and "Detector: motion" render. That's

consistent.

Now let me scrutinize the key behavioral concerns. Let me look at how App.tsx initializes `videoId` and whether the wizard/ingest gating interacts with deep-link loads, and check the `log` lib signature.

16. user (2026-06-22T03:46:16.646Z)

```
40      { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
41      { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
42    ];
43
44    type LoadState =
45      | { status: "demo" }
46      | { status: "loading" }
47      | { status: "loaded"; count: number }
48      | { status: "error"; error: string };
49
50    export default function App() {
51      const { t, i18n } = useTranslation();
52      // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
53      // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
54      // without a reload (it also rewrites ?video= so a refresh is stable).
55      const [videoId, setVideoId] = useState<string | null>(
56        () => new URLSearchParams(window.location.search).get("video"),
57      );
58
59      const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60      const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
61      const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
62      const [reelName, setReelName] = useState("highlights");
63      const [build, setBuild] = useState<BuildState>({ status: "idle" });
64      // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
65      const [wizardDone, setWizardDone] = useState(isWizardDone);
66
67      // Display order is separate from selection: a query can re-sort the cards while the
shared
68      // selection set (sel) stays the single source of truth for what lands in the reel.
69      const sel = useRallySelection(rallies, "all");
```

17. user (2026-06-22T03:46:17.018Z)

```
1      // Structured client logger ? the observability seed for the operator console
2      // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md §5). Every API call carries a CORRELATION ID so
a
3      // single user journey is traceable across the UI ? FastAPI ? worker. Logs are JSON-
shaped
4      // so they aggregate cleanly off a transient host. Keep this dependency-free.
5
6      export type LogLevel = "debug" | "info" | "warn" | "error";
7
8      /** A short, unique id to correlate all log lines (and the x-correlation-id header) for
one action. */
9      export function newCorrelationId(): string {
10        const c = (globalThis as { crypto?: Crypto }).crypto;
11        if (c && typeof c.randomUUID === "function") return c.randomUUID();
12        // Fallback for environments without crypto.randomUUID.
13        return `cid_${Date.now().toString(36)}_${Math.floor(Math.random() *
1e9).toString(36)}`;
```

```

14     }
15
16     export function log(level: LogLevel, event: string, fields: Record<string, unknown> =
17     {}): void {
18         const line = JSON.stringify({ ts: new Date().toISOString(), level, event, ...fields
19     });
20         if (level === "error") console.error(line);
21         else if (level === "warn") console.warn(line);
22         else console.log(line);
23     }
24 }

```

18. assistant (2026-06-22T03:46:26.097Z)

Now I have a complete picture. Let me analyze the key behaviors carefully.

****State machine analysis:****

- Initial: `loading=true, offline=false` ? `stepping=false`. Renders "checking" message. Bottom shows the single "Done" button (the `else` branch of `stepping`?), with onClick `finish`.

This is a real finding: ****during the loading phase****, the bottom button renders as `t("wizard.done")` ("Get started") and clicking it calls `finish()` ? marking the wizard done and closing it. So while capabilities are still being fetched, the user sees a "Get started" button that, if clicked, permanently dismisses the wizard before they ever see a single step. Let me confirm there's no guard.

Looking at lines 148-175: when `stepping` is false (which includes `loading===true`), it renders the single Done button. There's no distinction between loading and offline for that button. So during loading, "Get started" is live and finishes the wizard. Let me verify the loading window is realistic ? `getCapabilities` is a network call to `/api/capabilities`, so there's a non-trivial window.

Let me check whether the capabilities probe could be slow and whether there is any other concern. Let me also examine the `gemini` detector edge case ? when `default\_segmenter` is "gemini" but no key present.

Let me verify my reading of the disabled/Back logic and the offline path once more, and check the test for the loading-state button.

19. user (2026-06-22T03:46:27.375Z)

4:// no key-write endpoint yet, so we tell the user how to add the key (offline `motion` works with none).

```

83: const [loading, setLoading] = useState(true);
84: const [offline, setOffline] = useState(false);
116: const stepping = !loading && !offline; // the 3-step flow only when capabilities loaded
136: {stepping && (
141:   {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142:   {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143:   {stepping && step === 0 && <KeyStep caps={caps} />}
144:   {stepping && step === 1 && <ComputeStep caps={caps} />}
145:   {stepping && step === 2 && <VideosStep />}
149:   {stepping ? (
153:     disabled={step === 0}
155:     className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"

```

20. assistant (2026-06-22T03:46:42.806Z)

Confirmed. During `loading` (before the fetch resolves), the footer falls into the `else` branch (line 167-174) rendering the active "Get started" button wired to `finish()`. So a user who clicks while it's still "Checking what this device can do?" permanently dismisses the wizard. The content area says "Checking?" but the action button says "Get started" and is fully live ? the button misleads relative to the loading content. This is the "buttons mislead" race the

prompt asks about.

Now let me check whether there's a stitching/`gemini` mismatch concern and verify the `caps` empty-object degradation. Let me trace the optional chaining for a completely empty `{}` caps object:

```
- `KeyStep`: `caps?.gemini_key_present === true` ? `undefined === true` ? false ? shows  
"missing" guidance. Safe.  
- `ComputeStep`: `caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true` ? false.  
`caps?.gpu?.nvidia_smi_present === true` ? false. `caps?.default_segmenter ?? "motion"` ?  
"motion". Safe.
```

So a partial/empty caps degrades safely. Good. But note `caps` could also be `null` only via offline path (then `stepping` is false and steps don't render). In the stepping path, `caps` is whatever `getCapabilities` returned (could be `{}` if the engine returns an empty body, or even a non-object). The `?.` guards handle object/empty. If the JSON returned were `null` (engine returns literal `null`), `setCaps(null)` then `loading=false, offline=false` ? `stepping=true` and steps render with `caps=null` ? optional chaining still safe. Good.

Now, the `unmount during fetch` ? the `alive` guard at lines 87, 91, 96 properly prevents setState after unmount. No leak there. Good.

Let me check one more subtle thing: the offline "Done" button in the test uses `waitFor`. Why? Because in the offline branch, `finish()` is synchronous. Let me re-read ? actually that's just test style. Let me verify there's no double-render of two "Get started" buttons. In offline state, only the footer Done button renders ("Get started"), and the content area shows the offline message. The header also has a "Skip" button. Fine.

Now let me check: is there a concern that the **loading state has no way to NOT mislead**, plus is the loading "Get started" button actually reachable? The fetch is to localhost engine; in the bundled appliance it's same-origin and fast, but on a cold engine start or slow box it's a real window. Let me confirm the test doesn't cover the loading-state button (it doesn't ? tests only click after `findByText` resolves the loaded content).

Let me verify there are no other locale files missing keys (degradation: missing key falls back to en, fine) and check the `aria-live` + role correctness quickly, then verify the full diff has nothing else.

21. user (2026-06-22T03:46:45.778Z)

```
=== kn ===  
keys: ['back', 'checking', 'compute', 'done', 'intro', 'key', 'next', 'offline', 'skip', 'step',  
'title', 'videos']  
step has {n}: True  
detector has {name}: True  
=== hi ===  
keys: ['back', 'checking', 'compute', 'done', 'intro', 'key', 'next', 'offline', 'skip', 'step',  
'title', 'videos']  
step has {n}: True  
detector has {name}: True  
=== es ===  
keys: ['back', 'checking', 'compute', 'done', 'intro', 'key', 'next', 'offline', 'skip', 'step',  
'title', 'videos']  
step has {n}: True  
detector has {name}: True  
=== da ===  
keys: ['back', 'checking', 'compute', 'done', 'intro', 'key', 'next', 'offline', 'skip', 'step',  
'title', 'videos']  
step has {n}: True  
detector has {name}: True  
=== id ===  
keys: ['back', 'checking', 'compute', 'done', 'intro', 'key', 'next', 'offline', 'skip', 'step',  
'title', 'videos']  
step has {n}: True  
detector has {name}: True
```

22. assistant (2026-06-22T03:46:49.993Z)

All locales have all wizard keys with correct ICU placeholders. Good.

Now let me run the test suite to confirm green (and confirm my loading-button finding isn't covered), and run lint/typecheck per the pre-LGTM gate norms.

23. user (2026-06-22T03:46:57.668Z)

```
stdout | src/components/SetupWizard.test.tsx > SetupWizard >
reflects a connected key + a GPU/ffmpeg-ready box
{"ts":"2026-06-22T03:46:57.016Z","level":"info","event":"api.response","cid":"47d72b33-826b-4f78-83c1-6a6ee5088446","method":"GET","url":"/api/capabilities","status":200}

stdout | src/components/SetupWizard.test.tsx > SetupWizard >
Skip closes + sets the flag without finishing the steps
{"ts":"2026-06-22T03:46:57.046Z","level":"info","event":"api.request","cid":"ba8584eb-f0fe-4524-a416-f9c9ab466d47","method":"GET","url":"/api/capabilities"}

stdout | src/components/SetupWizard.test.tsx > SetupWizard >
Skip closes + sets the flag without finishing the steps
{"ts":"2026-06-22T03:46:57.050Z","level":"info","event":"api.response","cid":"ba8584eb-f0fe-4524-a416-f9c9ab466d47","method":"GET","url":"/api/capabilities","status":200}

stdout | src/components/SetupWizard.test.tsx > SetupWizard >
degrades gracefully when the engine is unreachable (offline) ? message + a Done escape
{"ts":"2026-06-22T03:46:57.064Z","level":"info","event":"api.request","cid":"f06c929e-7933-4cba-aaac-5db38b364970","method":"GET","url":"/api/capabilities"}

stderr | src/components/SetupWizard.test.tsx > SetupWizard >
degrades gracefully when the engine is unreachable (offline) ? message + a Done escape
{"ts":"2026-06-22T03:46:57.066Z","level":"error","event":"api.network_error","cid":"f06c929e-7933-4cba-aaac-5db38b364970","method":"GET","url":"/api/capabilities","error":"TypeError: Failed to fetch"}

stderr | src/components/SetupWizard.test.tsx > SetupWizard >
degrades gracefully when the engine is unreachable (offline) ? message + a Done escape
{"ts":"2026-06-22T03:46:57.066Z","level":"warn","event":"wizard.capabilities_unavailable","cid":"f06c929e-7933-4cba-aaac-5db38b364970","error":"ApiError: Network error calling GET /api/capabilities"}

? src/components/SetupWizard.test.tsx (6 tests)
296ms

Test Files   1 passed (1)
Tests       6 passed (6)
Start at    09:16:53
Duration    3.73s (transform 165ms, setup 24ms, collect 682ms, tests 296ms,
environment 1.95s, prepare 251ms)
```

24. assistant (2026-06-22T03:47:00.378Z)

Tests pass. Confirmed the loading-state misleading-button is NOT covered. Let me run the broader suite (App + a4 touch) and typecheck/lint to validate the change doesn't regress.

25. user (2026-06-22T03:47:17.059Z)

```
stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-22T03:47:16.563Z","level":"warn","event":"wizard.capabilities_unavailable","cid":"fe9be6d7-f06b-
```

```
4705-9842-067ff14dbeed", "error": "ApiError: Network error calling GET /api/capabilities"}
{"ts": "2026-06-22T03:47:16.563Z", "level": "warn", "event": "config.unavailable", "error": "ApiError:
Network error calling GET /api/config"}
```

? src/i18n/i18n.test.tsx (17 tests)

2085ms

? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys) 1000ms

Test Files 24 passed (24)

Tests 221 passed (221)

Start at 09:17:05

Duration 11.00s (transform 2.82s, setup 660ms, collect 15.17s, tests 11.03s,
environment 82.13s, prepare 10.75s)

26. assistant (2026-06-22T03:47:18.936Z)

All 221 tests pass. Let me run typecheck and lint.

27. user (2026-06-22T03:47:31.542Z)

=== LINT ===

Oops! Something went wrong! :(

ESLint: 10.5.0

ESLint couldn't find an eslint.config.(js|mjs|cjs) file.

From ESLint v9.0.0, the default configuration file is now eslint.config.js.

If you are using a .eslintrc.* file, please follow the migration guide
to update your configuration file to the new format:

<https://eslint.org/docs/latest/use/configure/migration-guide>

If you still have problems after following the migration guide, please stop by
<https://eslint.org/chat/help> to chat with the team.

28. assistant (2026-06-22T03:47:33.809Z)

Typecheck passed clean (no output). The lint config isn't at the web root ? let me find how the
repo runs lint.

29. user (2026-06-22T03:47:35.349Z)

```
{
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit",
  "test": "vitest run",
  "test:coverage": "vitest run --coverage"
}
```

30. assistant (2026-06-22T03:47:43.748Z)

No lint script in this package ? lint isn't part of this web package's gate (typecheck + tests
are). Typecheck and the full suite are green.

I have all I need. Let me compile findings. The main concrete bug is the misleading "Get
started" button during the loading window. Let me double-check one secondary point: the `gemini`
detector-without-key inconsistency ? if `default\_segmenter === "gemini"` but `gemini_key_present
=== false` (the engine could return that), the KeyStep would say "No AI key yet" while
ComputeStep says "Detector: gemini". That's a possible internal inconsistency, but it's the

engine's reported state, low severity. Let me note it as a nit.

Let me also verify the `aria-live="polite"` region: it wraps both the loading text and the step content. When the user clicks Next, the whole step content swaps inside an aria-live region, which would cause screen readers to announce the entire new step on every navigation ? acceptable but worth a nit. I'll keep findings focused on the concrete ones.

Let me verify the loading-window finding is genuinely reachable by confirming there's no `disabled`/conditional on that footer Done button that I missed ? re-reading lines 167-175: no disabled, no loading guard. Confirmed.

31. user (2026-06-22T03:48:01.669Z)

Structured output provided successfully

32. assistant (2026-06-22T03:48:12.933Z)

Review complete. I scrutinized SetupWizard.tsx, its test, App.tsx wiring, the API client/types, the i18n config, and all six locale catalogs, then ran the full web suite (221 tests green) and typecheck (clean).

The one concrete bug worth fixing:

- **major ? misleading finish button during the loading window**
(`web/src/components/SetupWizard.tsx:148-175`): while `getCapabilities()` is in flight (`loading=true, offline=false`), `stepping` is false, so the footer renders an active "Get started" button wired to `finish()`. The content area only says "Checking?", but clicking the button permanently sets `WIZARD\_DONE\_KEY` and closes the wizard ? sending a new user past the wizard forever without seeing step 1. The offline branch intentionally offers this escape; the loading branch should not. No test covers this path. Fix: gate the single finish button on `offline` only, render nothing actionable while `loading`.

The other concerns the prompt raised checked out clean:

- **Race / unmount-during-fetch**: the `alive` guard (lines 87/91/96) correctly blocks setState after unmount; no leak.
- **localStorage flag**: `isWizardDone()` and `finish()` both wrap access in try/catch; private-mode degrades to "show it, harmless" ? correct.
- **Optional chaining over caps**: an empty `{}` or even literal `null` caps degrades safely ? KeyStep falls to "missing", ComputeStep falls to motion/no-ffmpeg/no-gpu, no crash.
- **Back/Next/Done/Skip + isLast/disabled**: Back disabled at step 0, Next clamps to last, isLast routes to finish ? all correct. ICU placeholders (`{n}`, `{name}`) are present and valid in all six locales.

Two nits filed: a possible step-1 vs step-2 contradiction when `default\_segmenter="gemini"` but no key present, and the `aria-live` region re-announcing the full step on each navigation. Both low-risk for v1.